Cluster Analysis Notes: Clustering Methods

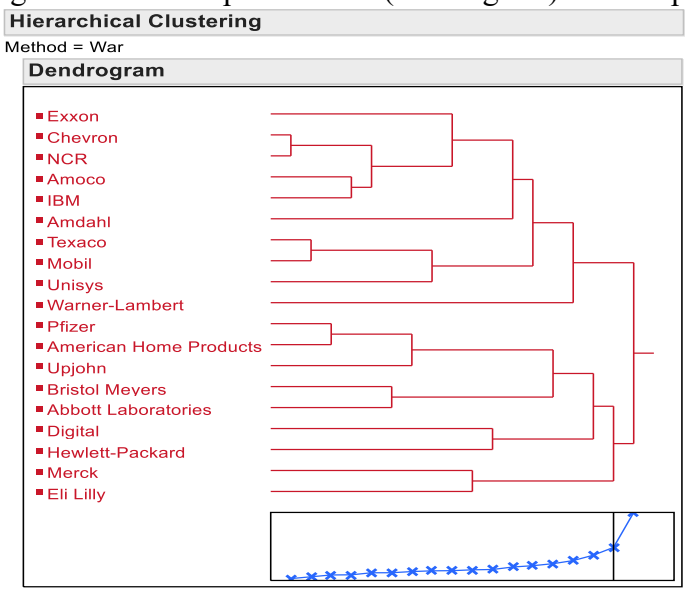
In this section of the Notes, I discuss the two main groups of clustering methods: hierarchical agglomerative and iterative partitioning (in particular, *K*-means clustering).

Hierarchical Agglomerative Methods

These are the most-used clustering methods. They all begin in a bottom-up manner with each case in its own cluster and proceed to join cases together, one at a time, to form larger clusters, eventually ending with all cases in one super-cluster. So there are *n* steps. At the end, the intermediate history of the joining (agglomeration) can be displayed as a **TREE**, with the cases listed on the horizontal axis at the bottom (or left side), with a vertical line leading up from each case until it joins with the line of another case or cluster, the height of the tree indicating the degree of (dis)similarity when the join occurs. [See Figure 3 for an example.] So the "tree" is upside-down (or sideways) with leaves at the bottom (or left side) and root at the top (or right side). In fact, the arboreal vocabulary is retained and mixed with genealogical vocabulary: A **LEAF** is a case; a **ROOT** is the super-cluster at the top (right side) consisting of all the leaves; a **BRANCH** is a cluster containing more than one, but not all, cases; a **NODE** is a leaf, a branch, or a root; if cluster A is formed by joining clusters B and C, then A is the **PARENT** of B and C, which are, of course, the **CHILDREN** of A. So a leaf is a node with no children; a root is a node with no parent. If every cluster has at most two children, the tree is **BINARY**.

The degree of relatedness of two cases is indicated by their most recent common "ancestor" (cluster in which both are members) in the resulting hierarchy. Each of the *n* steps in the agglomeration represents a possible solution to the two major clustering problems (how many clusters and how to assign cluster membership). Presumably, the solution that puts all cases in separate clusters (at the bottom [left] of the tree) and the solution with all cases in one super-cluster (at the top [right] of the tree) are not acceptable. Solutions that are in-between the top and the bottom can be examined and tested for their reasonableness.

Figure 3. An example of a tree (dendrogram): 19 companies in Forbes 500



The tree in Figure 3 is read from left to right. At the first step, Chevron and NCR are found to be the two most similar companies and are joined together in a cluster. At the second step, Texaco and Mobil are the most similar and are joined. Eventually, clusters start to join in larger clusters. For example, in step 5 the Chevron/NCR cluster that was formed in step 1 joins the Amoco/IBM cluster that was formed in step 4. The joins continue until in step 18, two large clusters of 10 and 9 companies join to form the root cluster of all 19 companies. The tree can be cut by a vertical line at any point between the root and the leaves to yield a solution. The branches that remain intact at the cut point become the clusters for the solution. For example, if the tree is cut vertically just to the left of the last joining, then there will be two large intact branches of 9 and 10 companies, respectively.

Next, I will briefly discuss some of the methods used in hierarchical agglomerative clustering. They are all binary; by default they all define the distance between cases as Euclidean distance between the profiles of the cases; they differ in terms of how they use pairwise case distances to determine the distances between clusters. Each method begins with an n x n (dis)similarity matrix populated with the pairwise (dis)similarities (distances) between each pair of cases.

(1) **Single Linkage** ("**Nearest Neighbor**"). This hierarchical agglomerative method begins with each case in its own cluster and joins the two most similar (closest) cases into a new cluster, replacing the two former cases. After the join, the (dis)similarity between the new cluster and remaining ones must be redefined and calculated. In single linkage, the (dis)similarity between the new cluster and a remaining entity is defined to be the (dis)similarity between the *nearest* members of the entity and the cluster – i.e., the greatest pairwise similarity (least dissimilarity) between the entity and any member of the new cluster. This means that the algorithm can continue to use the initial (dis)similarity matrix – no need to recompute its values – one just needs to keep track of which rows have been clustered. At the next step, the two most similar entities from the new set of clusters are joined, and the (dis)similarities between remaining entities and the new entity are recalculated. The process continues until all entities have been joined in the super-cluster at the root. This method tends to do the worst in simulation studies. It tends to form elongated clusters of unequal numbers of cases (e.g., two clusters of 89 cases and one cluster of one case).

(2) **Complete Linkage** ("**Farthest Neighbor**"). Like single linkage, except that (dis)similarity is the distance between the *farthest* members of two entities. So a joining entity must be close to ALL members of the entity to be joined. In complete linkage, the (dis)similarity between the new cluster and a remaining entity is defined to be the least pairwise similarity (greatest dissimilarity) between the entity and any member of the new cluster. Again, like single linkage, this means that the algorithm can continue to use the initial similarity matrix – no need to recompute its values – one just needs to keep track of which rows have been clustered. The complete linkage method tends to find compact hyper-spherical clusters of highly similar cases.

(3) **Average Linkage** ("**Average Neighbor**"). Like single and complete linkage, average linkage must (re)define the distance between existing entities and an entity newly formed by joining. In average linkage, the (dis)similarity between the new cluster and a remaining entity is defined to be the mean (average) pairwise similarity (mean dissimilarity) between the entity and the members of the new cluster. Unlike single linkage and complete linkage, the similarity

matrix must be recomputed after each step. This method was proposed as an antidote to the extremes of single and complete linkage. It tends to find clusters of roughly equal variance. Average linkage and Ward's method tend to do best in simulation studies.

(4) **Centroid Method**. The **centroid** of a cluster is the vector of means of the *p* variables of the cases in the cluster. The centroid method is like single linkage, except that the (dis)similarity between two clusters is defined to be the squared Euclidean distance between the two centroids of the two clusters. When two clusters are joined, the centroid of the new cluster becomes the reference point for recalculating (dis)similarities.

(5) **Ward's Method** [the default in `hclust` in R] Like single linkage, except that at each step, clusters are joined that result in the least increase in the within-cluster variance.[1] To gain an intuitive understanding of what Ward's method tries to do, look at Figure 1, which shows two clusters. Ward's method tries to choose the clusters to make the points within each cluster as close to each other as possible and as far from the other cluster as possible. In Figure 1, the points within the lower left cluster are closer to each other, on average, than they are to the points in the upper right cluster, and vice-versa. The lower left cluster is clearly separated from the upper right cluster. One can measure how close the points within each cluster are to each other by their variance: The smaller the variance within a cluster, the closer its points are to each other. In the most extreme case, the variance within a cluster could be zero if the points are all equal to the mean within the cluster. The bigger the variance within a cluster, the farther apart its points are. In Figure 1, the points within the lower left cluster clearly have less within-cluster variance than the points in the upper right cluster.

How can the variance within a cluster be calculated? The next several paragraphs explain how to do this. This material explains the criterion for Ward's method. But it also provides a criterion to evaluate how well any and all clustering methods work – not just Ward's method. By way of preview, it turns out that an "R-square" can be calculated for any proposed clustering to represent the explanatory power of the proposed clustering.

Suppose that X_{ikl} is the value of case *l* on variable *k* in cluster *i*. Suppose also that n_i is the number of cases in cluster *i*, and that there are *c* clusters. Then the within-cluster sum-of-squares for variable *k* in cluster *i* is

$$\sum_{l=1}^{n_i}(X_{ikl} - \overline{X}_{ik.})^2 \quad \textbf{[Exp 1]}$$

where $\overline{X}_{ik.}$ is the mean of all of the kth variable values that are in cluster *i*. So $\sum_{l=1}^{n_i}(X_{ikl} - \overline{X}_{ik.})^2$ is the total squared deviation between this mean and the kth variable's values of the cases that are in cluster *i*. So $\sum_{l=1}^{n_i}(X_{ikl} - \overline{X}_{ik.})^2$ is a natural measure of how big cluster *i* is in terms of variable *k* – i.e., how close these points are to each other. In fact, it would be the variance of variable *k* in cluster *i* if it were divided by n_i -1. Converting this into a variance is not a necessary step since it is just adjustment by a constant. We can use the sum of squared deviations itself as the criterion.

[1] Actually, the within-cluster sum of squares – the variance is the sum of squares divided by the degrees of freedom.

But we are not finished. $\sum_{l=1}^{n_i}(X_{ikl}-\bar{X}_{ik\cdot})^2$ is a measure that can be calculated for each of the c clusters. Let us add them up over all clusters to get an overall measure of closeness for variable k for all c clusters =

$$\sum_{i=1}^{c}\sum_{l=1}^{n_i}(X_{ikl}-\bar{X}_{ik\cdot})^2 \quad \textbf{[Exp 2]}$$

But we are still not finished. We can calculate this for each of the variables. If there are p variables, then the total within-cluster variance (sum-of-squares) is

$$\sum_{k=1}^{p}\sum_{i=1}^{c}\sum_{l=1}^{n_i}(X_{ikl}-\bar{X}_{ik\cdot})^2 \quad \textbf{[Exp 3]}$$

which is called the **within-cluster variance** (more precisely: the within cluster sum of squares). This is the criterion that Ward's method seeks to minimize from step to step in the clustering algorithm. Ward's method tends to create hyper-spherically-shaped clusters having roughly equal numbers of cases. Ward's method and average linkage tend to do best in simulation studies.

Extension: At the same time that Ward's method minimizes the within-cluster sum-of-squares, it maximizes the between-cluster sum-of-squares, thus obtaining maximal spread between the clusters. There is a mathematical identity that illuminates this relationship. For variable k:

$$\sum_{i=1}^{c}\sum_{l=1}^{n_i}(X_{ikl}-\bar{X}_{\cdot k\cdot})^2 = \sum_{i=1}^{c}n_i(\bar{X}_{ik\cdot}-\bar{X}_{\cdot k\cdot})^2 + \sum_{i=1}^{c}\sum_{l=1}^{n_i}(X_{ikl}-\bar{X}_{ik\cdot})^2 \quad \textbf{[Exp 4]}$$

where $\bar{X}_{\cdot k\cdot}$ is the grand mean of all of the data for variable k. **Exp 4** says that the total variability of all of the data for variable k (the left-hand side) can be broken down into a measure of the deviations of the cluster means from the grand mean + the deviations of the data from the cluster means (**Exp 2**).

Then add up **Exp 4** over all p variables to get the identity:

$$\sum_{k=1}^{p}\sum_{i=1}^{c}\sum_{l=1}^{n_i}(X_{ikl}-\bar{X}_{\cdot k\cdot})^2 = \sum_{k=1}^{p}\sum_{i=1}^{c}n_i(\bar{X}_{ik\cdot}-\bar{X}_{\cdot k\cdot})^2 + \sum_{k=1}^{p}\sum_{i=1}^{c}\sum_{l=1}^{n_i}(X_{ikl}-\bar{X}_{ik\cdot})^2 \quad \textbf{[Exp 5]}$$

Total Sum-of-Squares= **Between-cluster Sum-of-Squares** + **Within-cluster Sum-of Squares**

This can be written simply as **TSS** = **BSS** + **WSS**. The identity says that the total variability of all of the data (**TSS**) can be broken down into a measure of the deviations of the cluster mean vectors from the grand mean vector (**BSS**) + the deviations of the data vectors from the cluster mean vectors (**WSS**).

Once the data are in hand, **TSS** is fixed. If you change the clusters, **TSS** does not change, but **BSS** and **WSS** do change. Therefore, if you change the clusters so that **WSS** decreases, then **BSS** must increase. At the beginning of the hierarchical process for Ward's method, each point is in its own singleton cluster; WSS = 0, and BSS is at its maximum and equals TSS. As the agglomeration proceeds, WSS increases (but slowly, since Ward's method minimizes its growth) and BSS decreases. At the end of the agglomeration, all points are in one super-cluster, with BSS = 0 and WSS is at its maximum and equals TSS.

From the identity **TSS** = **BSS** + **WSS**, further insight can be gained into Ward's method – and all other clustering methods as well. This identity can answer the question, How much of the X variables can be explained by the clustering of the cases? The question is answered by regressing the X's on the clustering. That sounds like a strange kind of regression. To see how it works, let us create c indicator variables – one for each cluster – to use as predictor variables in

the regression. Define $I_1 = 1$ if a case is in cluster 1, $I_1 = 0$ if the case is not in cluster 1; $I_2 = 1$ if a case is in cluster 2, $I_2 = 0$ if the case is not in cluster 2; …; $I_c = 1$ if a case is in cluster *c*, $I_c = 0$ if the case is not in cluster *c*. Then the collection of *c* indicator variables represents the clustering. Collectively, they have all of the information that there is about the clustering.[2]

Now suppose we ask, How much information about the *X* variables is conveyed by the clustering? That is the same as asking, How much information about the *X* variables is conveyed by the indicator variables? The latter can be answered by regressing the *X*'s on any *c* – 1 of the indicators $I_1,...,I_c$.[3] It turns out that the R-square for this regression can be computed as **BSS**/**TSS**. So BSS is the sum-of-squares for regression, and WSS is the error sum-of-squares. The insight that we get from this about Ward's method is that Ward's method tries to maximize the R-square – the explanatory power – of the clustering as clusters are combined during agglomeration. In selecting a solution (a cut point on the dendrogram), the user must decide on an appropriate trade-off between explanatory power (R-square) and parsimony (number of clusters). This recalls similar tradeoffs in statistical model building in general, as the gain in explanatory power from adding a predictor must be weighed against the loss of parsimonious explanation; one wants a parsimonious model with high explanatory power. In CA, the ideal situation is to have a few clusters that still have high explanatory power. In regression, that corresponds in general to wanting a few predictors that have high explanatory power. We should note that such considerations can be applied to all hierarchical agglomerative clustering methods – indeed to *all* clustering methods. The BSS/TSS ratio can be calculated for any proposed cluster solution and thus evaluate its explanatory power.

As a further methodological note, the regressions of the *X*'s on the cluster indicators amount to **analysis of variance** (**ANOVA**),[4] which is the statistical technique for regression with categorical predictors. There is no real difference between ANOVA and regression with indicator predictors. But the two methodologies developed historically on different paths, with different terminology (BSS instead of regression sum of squares, WSS instead of error sum of squares, factors instead of predictors, BSS/TSS instead of R-square, etc.)

(6) **Fast Ward** and (7) **Hybrid Ward** are variations on Ward's method that are designed to speed up the algorithm. Nearest neighbor joining is used initially to reduce the size of the matrix that is passed to Ward's algorithm. Then regular Ward is used during the last few hundred joins. In R's `hclust` routine, there are two options for Ward's method, `ward.D` and `ward.D2`, with `ward.D2` corresponding to Ward's original algorithm.

---

[2] Actually, any subset of *c* – 1 of them has all of the cluster information. The *c* indicator variables are collinear, so once you know any *c* – 1 of them, you know the remaining one.

[3] One of the indicators must be omitted to avoid a multicollinearity.

[4] When analysis of variance is done for each of *p* response variables and the results of the ANOVAs are combined, as they are in the formula TSS = BSS + WSS, the procedure is called **multiple analysis of variance** (**MANOVA**).

Iterative Partitioning Methods: *K*-means Clustering

The major alternative to hierarchical clustering methods is iterative partitioning. Most iterative partitioning methods begin by dividing the cases into a number of user-specified preliminary clusters. This is the first (preliminary) generation of clusters. To get the second generation, reference points are calculated for each cluster in the first generation of clusters. These reference points are usually the centroids of the clusters – but need not be.[5] The reference points become the "seeds" for growing the second generation. Each case is then re-assigned to one of the seeds, as determined by the reassignment method. Often, the reassignment is to the nearest seed in terms of Euclidean distance. The result is a partitioning of the cases into a second generation of clusters. The second generation has the same number of clusters as the first generation, but some cases may switch membership to other clusters. Then reference points for the second generation are computed and become the seeds for growing the third generation. Each case is then re-assigned per the reassignment method, and the process iterates until no case changes cluster membership, or until the limit on the number of passes is reached. The most well-known and most commonly used of these iterative partitioning methods is ***k*-means clustering**.

In all iterative partitioning methods, there are three major decisions to make:

1) Initial partition. This can be determined by pre-specifying a set of seeds, or by determining the preliminary partition. Both of these can be done randomly or by a method.
2) Seed selection. This is how the seeds (cluster reference points) for the next generation are to be chosen. Each cluster in the current generation produces a seed for the next generation. A popular method is to choose the centroid (mean vector) of each cluster.
3) Reassignment method. This is the way to reassign cases for the next generation during each pass. A common method is reassignment to nearest centroid, called *k*-means clustering.[6]

Simulation studies show that the major cause of inferior performance in iterative partitioning is a poor initial partition. This issue is often addressed by repeating the clustering a number of times with different randomly generated initial partitions.[7]

***K*-means clustering**. The objective of *k*-means clustering is to partition the data rows into a given number (*k*) of clusters in such a way that the within-cluster sum-of-squares (WSS – see discussion of Ward's method in Part 1 of the Cluster Analysis notes) is minimized. In its objective, *k*-means clustering resembles Ward's method. Both methods seek to minimize the WSS. However, Ward's method operates hierarchically and agglomeratively, whereas *k*-means operates iteratively. Ward's method produces a solution for each possible number of clusters, whereas *k*-means produces a solution for only one given number (*k*) of clusters. *K*-means begins with an initial set of *k* cluster "seeds" spread more or less uniformly across the data space. These seeds can be user specified, picked at random, or selected by some algorithm. They are often cases themselves, but need not be. Then, in the first pass, each case is assigned to the closest

---

[5] For example, an alternative to the centroid is the **medoid**, which is always a case in the data set. Medoids may be used in instances like graphs or images in which a centroid cannot be defined or is not representative.

[6] If assignment is made to medoids, the technique is called ***k*-medoid clustering**.

[7] For example, the KMEANS function in R has the option NSTART=<*n*> that generates *n* initial partitions and uses the best one.

seed. This results in *k* preliminary clusters. Then the centroid (mean) of each cluster is calculated. The *k* centroids become the new seeds. Another pass is made through the cases. During the pass, each case is (re)assigned to the closest centroid/new seed/mean. A revised set of *k* clusters results.[8] Centroids are calculated yet again, and the process is iterated until no case changes cluster membership – or until the limit on number of passes is reached.

Determining the Number of Clusters

This remains an unsolved problem in theoretical statistics. None of the proposed solutions is widely accepted. The reasons for this:

1) A suitable formulation of a null hypothesis to capture the meaning of "there are no clusters" is elusive.
2) A suitable formulation of alternative hypotheses is elusive.
3) Those hypotheses that can be tested are uninformative.
4) Those hypotheses that are most meaningful have test statistics with intractable distributions.
5) One might think that F tests based on (M)ANOVA (such as those found in **R**) could be applied to test whether the cluster means are significantly different. The problem is that such tests are invalid for testing differences between clusters. The reason is that CA is *structure-imposing*: CA will find clusters even if none exist. Consequently, such F tests are almost always misleadingly significant.

In spite of the lack of a theoretical solution for the problem of determining the number of clusters, a large number of heuristic approaches have been proposed. Many of these seem to work well in practice with real data of certain types. Refining existing heuristic approaches and developing new approaches remains an active area of statistical/machine learning research.

I will now briefly discuss some of the more widely used clustering heuristics and offer some advice and observations on determining the number of clusters:

1) Always pay attention to the underlying practice or theory of your subject. If practice or theory suggests a certain number of clusters or if there is some "natural" number of clusters, then be sure to include that number among the possibilities that you check out. For example, some biologists propose that all cellular species can be partitioned into three "Empires": Bacteria, Archaea, and Eukaryota. Then a CA of species should include $k = 3$ as a possible solution for the number of clusters.

2) The "Elbow" method. Based on the same idea as a "scree" plot to determine the number of factors in factor analysis, the Elbow method proposes to graph some measure of cluster (un)worthiness vertically against the number of clusters horizontally. Then pick the number of clusters at the "elbow" in the plot – that is, the point where the plot flattens out (if it does). For example, plot WSS(k) vs k, where k is the number of clusters and WSS is the within-sum-of-squares for k clusters. WSS is a measure of the (dis)similarity of the cases within

[8] This explains the origin of the name "*k*-means": data points are assigned to the closest of *k* means (centroids).

clusters.[9] If there is an "elbow" in the plot, then at that k, WSS(k) will differ substantially from WSS(k-1) but will not differ much from WSS(k+1). That means that adding another cluster to k will not improve the similarity of cases within clusters by very much.[10] So if WSS stops changing much, then the "worthiness" of the clustering has plateaued.
The Elbow method is subjective and is not used as a serious tool to decide the number of clusters. Rather, it may suggest possibilities and provide confirmation. In addition, there often is no "elbow".

3) *K*-means clustering requires you to choose the number of clusters (k) in advance. If you are not sure how many there are, you could pick a k and also try values near k like k-2, k-1, k+1, k+2. The models could be distinguished on the basis of differences in R-square, adjusted R-square, AIC, or BIC.

4) SAS Institute developed the **cubic clustering criterion** (**CCC**) to help decide the number of clusters. A possible solution is indicated for k's that are local peaks in the CCC when plotted against the number of clusters k.

5) Hierarchical agglomerative methods do not require you to choose the number of clusters in advance. Moreover, hierarchical methods give you a complete genealogy or family history of how the clustering for one k is related to every other k. In k-means clustering, the solution for one k is not necessarily a descendant generation for k-1, nor a parent generation for k+1. So if the relationship between clustering generations is important to your subject matter, k-means clustering may not be an appropriate technique. Elephants did not descend from dinosaurs.

6) Average silhouette method. The concept **silhouette** is a measure of how similar a case is to the other cases in its own cluster, relative to how dissimilar the case is to cases in all other clusters. Intuitively, we want each case to be similar to other cases in its own cluster but dissimilar to cases in all other clusters. So given a case i, we calculate a measure of similarity $a(i)$ to the other cases in its *own* cluster. We also calculate a measure of dissimilarity $b(i)$ to the cases in all *other* clusters. We would like for the difference $b(i) - a(i)$ to be large. Not just for the one given case i, but for all cases collectively. We prefer a clustering in which this happens. So choose k to maximize the collective difference.
To make this idea specific, for a given case i, define $a(i)$ to be the mean distance between case i and all *other* cases in the same cluster. Let $b(i,j)$ be the mean distance between case i and all cases in cluster j. *Define* $b(i) = \min_j b(i,j)$ to be the *minimum* of these mean distances over all j.

The **silhouette** of case i is defined to be a standardized version of the difference $b(i) - a(i)$:

[9] The smaller the WSS, the more similar the cases within clusters.
[10] For the same reason, the explanatory power BSS (hence, R-square) of the clustering will cease to change much at that k. This is because WSS + BSS = a constant (TSS).

$s(i) =$	0	if case i is a singleton cluster
	$\dfrac{b(i)-a(i)}{\max\{a(i),b(i)\}}$	Otherwise

It follows that $-1 \le s(i) \le +1$. If $s(i)$ is close to +1, then $b(i)$ dominates $a(i)$, so the distance from i to neighboring cluster cases is large and the distance from i to its own cluster cases is small. Thus, case i is well-placed in its cluster.
The reverse holds if $s(i)$ is close to -1: If $s(i)$ is close to -1, then $a(i)$ dominates $b(i)$, so the distance from i to neighboring cluster cases is small and the distance from i to its own cluster cases is large. Thus, case i is poorly-placed in its own cluster, for it is more like cases in other clusters than it is like cases in its own cluster.
The **average silhouette** for a given clustering k is the mean of $s(i)$ over all cases i in the dataset. A good clustering should have a large average silhouette. The **average silhouette method** plots the average silhouette vertically against the number of clusters k horizontally. The k that yields the maximum average silhouette is suggested as the solution for the number of clusters.

7) The Gap Statistic method. The "Gap Statistic" estimates the difference (gap) between the expected value that WSS(k) would have if there were no clusters and the observed value of WSS(k). The **Gap Statistic method** suggests the k that maximizes the gap as the solution for the number of clusters.
The intuition for this idea is that WSS(k) should be large if there are no clusters but small if there are clusters: For, if there are no clusters, then the clustering will arbitrarily force cases together that are just as separated inside a cluster as they are outside; whereas if there are clusters, then the clustering will tend to put cases together that are like each other, hence are closer together within a cluster than outside. The stronger the clustering, the wider the Gap between the within-cluster separation under the hypothesis of no clusters and the within-cluster separation under the hypothesis that there are clusters.
To make this idea specific, define a **reference distribution** (without clusters) as a uniform distribution on the range of each variable used in the clustering: Uniform on [min(X_j), max(X_j)] for each j. This is a flat, non-"lumpy" probability distribution. Clearly, with a flat probability, there can be no clusters in data drawn from this distribution except by chance. The reference distribution represents a null hypothesis of "there are no clusters".
Randomly simulate B samples of size n each from the reference distribution. For each sample, calculate *log* WSS(k) for $k = 1, \ldots, k^*$, where k^* is the maximum value of k that you wish to contemplate. For each k, average the values of *log* WSS(k) over the B samples. This average is a simulated estimate of the expected value of *log* WSS(k) for the reference (clusterless) distribution. Define the **Gap Statistic** = Average {*log* WSS(k)} – *log* WSS(k), for each k. Note that in this expression, Average {*log* WSS(k)} is calculated using the simulated clusterless data, whereas *log* WSS(k) is calculated with the actual (potentially clustered) data.
The **Gap Statistic method** is to choose k as the value that maximizes the Gap Statistic. A refinement on this procedure is to require a clear signal that the chosen k is the one by having a Gap Statistic that exceeds that of neighboring k's by a given amount.